Our experiences have shown that most project teams create a memo for the project files that provides additional detail about the network model. This information is helpful to the people who are responsible for creating the hardware and software specifications (described next in this chapter) and who will work more extensively with the infrastructure development. This memo can include special issues that affect communications, requirements for hardware and software that might not be obvious from the network model, or specific hardware or software vendors or products that should be acquired.

The primary purpose of the network model diagram is to present the proposed infrastructure for the new system. The project team can use the diagrams to understand the scope of the system, the complexity of its structure, any important communication issues that might affect development and implementation, and the actual components that need to be acquired or integrated into the environment.

HARDWARE AND SYSTEM SOFTWARE SPECIFICATIONS

The time to begin acquiring the hardware and software that will be needed for a future system is during the design of the system. In many cases, the new system will simply run on the existing equipment in the organization. Other times, however, new hardware and software must be purchased. The *hardware and software specification* is a document that describes what hardware and software are needed to support an application. The actual acquisition of hardware and software should be left to the purchasing department or the area in the organization that handles capital procurement. However, the project team writes the hardware and software specification to communicate the project needs to the appropriate people. There are several steps involved in creating the document. Figure 11-12 shows a sample hardware and software specification.

First, we need to define the software that will run on each component. This usually starts with the operating system (e.g., Windows, Linux) and includes any special-purpose software on the client and servers (e.g., Oracle database). This document should consider any additional costs, such as technical training, maintenance, extended warranties, and licensing agreements (e.g., a site license for a software package). The listed needs are influenced by decisions that are made in the other design activities.

Specification	Standard Client	Standard Web Server	Standard Application Server	Standard Database Server
Operating System	• Windows • Chrome	• Linux	• Linux	• Linux
Special Software	• Adobe Reader • VLC	• Apache	• Java	• Oracle
Hardware	• 816 GB Memory • 500 gig disk drive • Intel Core i6 • 2—22″ Monitors	• 32 GB Memory • 1TB disk drive • Intel Core i7	• 32 GB Memory • 2–1 TB disk drives • Intel Core i7	• 64 GB Memory • 25 TB RAID • Intel Core i7
Network	• 100 Mbps Ethernet • WiFi	• 100 Mbps Ethernet	• 100 Mbps Ethernet	• 100 Mbps Ethernet

FIGURE 11-12 Sample Hardware and Software Specification

Second, we must create a list of the hardware that is needed to support the future system. With the advent of mobile computing (see Chapter 10), cloud computing (see earlier in this chapter), the IoT (see earlier in this chapter), and Green IT (see earlier in this chapter), this step is much more involved than it used to be. However, the low-level network model provides a good starting point for recording the project's hardware needs because each component on the diagram corresponds to an item on this list. In general, the list can include things like database servers, network servers, peripheral devices (e.g., printers, scanners), backup devices, storage components, and any other hardware component that is needed to support an application. At this time, you also should note the quantity of each item that will be needed.

Third, we must describe, in as much detail as possible, the minimum requirements for each piece of hardware. Typically, the project team must convey requirements like the amount of processing capacity, the amount of storage space, and any special features that should be included. Many organizations have standard lists of approved hardware and software that must be used; so in many cases, this step simply involves selecting items from the lists. Other times, however, the team is operating in new territory and is not constrained by the need to select from an approved list. This step becomes easier with experience; however, there are some hints that can help you describe hardware needs (see Figure 11-13). For example, consider the hardware standards within the organization or those recommended by vendors. Talk with experienced system developers or other companies with similar systems. Finally, think about the factors that affect hardware performance, such as the response-time expectations of the users, data volumes, software memory requirements, the number of users accessing the system, the number of external connections, and growth projections.

The last step to consider is to evaluate vendor proposals (see Chapter 7). The easiest way to do this is to create an *alternative matrix* (see Chapters 2 and 7). In this case, the evaluation criteria in the alternative matrix should include all architectural requirements, both optional and mandatory, and each criterion should be weighted. Some general criteria include CPU speed, bus speed, disk size, disk access time, cache size, cache speed, RAM size, RAM speed, data transfer rate, video RAM size and speed, monitor size, and printer resolution. Of course, in today's connected world, the networking hardware and software would also need to be specified, including routers, print servers, hubs, and switches. Mobile devices such as smartphones and tablets may be part of the physical architecture solution. Depending on the problem domain requirements, additional hardware and system software could be required, such as speech recognition and generation software and hardware, digitizing tablets, and possibly head-mounted displays, shutter glasses, force feedback pointing devices, and 3D printers.

Functions and Features What specific functions and features are needed (e.g., size of monitor, software features)?
Performance How fast does the hardware and software operate (e.g., processor, number of database writes per second)?
Legacy Databases and Systems How well does the hardware and software interact with legacy systems (e.g., can it write to this database)?
Hardware and OS Strategy What are the future migration plans (e.g., the goal is to have all of one vendor's equipment)?
Cost of Ownership What are the costs beyond purchase (e.g., incremental license costs, annual maintenance, training costs, salary costs)?
Political Preferences People are creatures of habit and are resistant to change, so changes should be minimized.
Vendor Performance Some vendors have reputations or future prospects that are different from those of a specific hardware or software system they currently sell.

FIGURE 11-13
Factors in Hardware and Software Selection

Each of these types of specialized devices has its own specialized evaluation criteria. In a nutshell, when creating a hardware and system software specification, most systems analysts find that they need help from IT and CS personnel.

Depending on the overall cost and size of the project, one thing that should be seriously considered is the use of a benchmark. A *benchmark* is essentially a sample of programs that would be expected to run on the new physical architecture. Even though benchmarks can be expensive to create, they tend to provide a more realistic picture of how the proposed physical architecture layer will perform.

When evaluating hardware, there is a set of issues that you should recognize.[16]

- Not only should you provide sample programs for the benchmarks, but you also need to provide actual data. Otherwise, the benchmark results could be misleading.
- You need to carefully review the mix of system software and hardware. For example, in many cases, Linux performs better on the same hardware when compared against Windows, but some applications might not be available under Linux. Consequently, there may be some trade-offs that should be considered.
- When considering adding additional hardware, be sure to evaluate the additional hardware based on marginal utility, not actual utility.
- Do not specify the physical architecture before you understand the problem domain requirements. This might seem obvious, but when you consider the time it takes for a mainframe computer, a large number of servers, or a large number of client machines to be specified, ordered, and delivered, it can be tempting to specify the hardware and system software prematurely. This could lead to either under- or over-specification.
- Recognize the reality of *Parkinson's Law*. From an IT perspective, Parkinson's Law implies that regardless of the users' real needs, their imagined needs will always fill up whatever capacity the system has. Consequently, it is imperative that the physical architecture layer design be based on the current and expected future architecture of the problem domain layer.
- Do not limit choices to a single vendor. This is especially true when you consider commodity hardware, such as displays, desktops, and department-size servers.
- Given the rate of technological change that is taking place in IT, consider leading-edge ideas. For example, even though tablet computers have been around for a while, the iPad™ was not on most people's radar. Today, it is considered to be a game changer when considering client-based hardware. Consequently, you really must stay up to date when it comes to the design of the physical architecture layer.

NONFUNCTIONAL REQUIREMENTS AND PHYSICAL ARCHITECTURE LAYER DESIGN

The design of the physical architecture layer specifies the overall architecture and the placement of software and hardware that will be used. Each of the architectures discussed before has its strengths and weaknesses. Most organizations use client–server architectures for cost reasons, so in the event that there is no compelling reason to choose one architecture over another, cost usually suggests client–server.

Creating a physical architecture layer design begins with the nonfunctional requirements. The first step is to refine the nonfunctional requirements into more-detailed requirements that

[16] Alton R. Kindred, *Data Systems and Management: An Introduction to Systems Analysis and Design*, 2nd Ed. (Englewood Cliffs, NJ: Prentice-Hall, 1980).

are then used to help select the architecture to be used (server-based, client-based, or client–server) and what software components will be placed on each device. In a client–server architecture, one also has to decide whether to use a two-tier, three-tier, or *n*-tier architecture. Then the nonfunctional requirements and the architecture design are used to develop the hardware and software specification.

Four primary types of nonfunctional requirements can be important in designing the architecture: operational requirements, performance requirements, security requirements, and cultural/political requirements. Furthermore, each of these requirements must be fully verified and validated.

Operational Requirements

Operational requirements specify the operating environment(s) in which the system must perform and how those might change over time. This usually refers to operating systems, system software, and information systems with which the system must interact, but on occasion it also includes the physical environment if the environment is important to the application (e.g., it's located on a noisy factory floor, so no audible alerts can be heard). Figure 11-14 summarizes four key operational requirement areas and provides some examples of each.

Technical Environment Requirements *Technical environment requirements* specify the type of hardware and software system on which the system will work. These requirements usually focus on the operating system software (e.g., Windows, Linux, and Mac OS), database system software (e.g., Oracle), and other system software (e.g., Firefox). In today's distributed world, issues related to mobile computing (see Chapter 10), cloud computing (see the earlier section in this chapter), the IoT (see earlier section in this chapter), and Green IT (see the

Type of Requirement	Definition	Examples
Technical Environment Requirements	Special hardware, software, and network requirements imposed by business requirements	• The system will work over the Web environment with Internet Explorer. • All office locations will have an always-on network connection to enable real-time database updates. • A version of the system will be provided for customers connecting over the Internet via a tablet or smartphone.
System Integration Requirements	The extent to which the system will operate with other systems	• The system must be able to import and export Excel spreadsheets. • The system will read and write to the main inventory database in the inventory system.
Portability Requirements	The extent to which the system will need to operate in other environments	• The system must be able to work with different operating systems (e.g., Linux, Mac OS, and Windows). • The system might need to operate with handheld devices, such as Android and Apple iOS devices.
Maintainability Requirements	Expected business changes to which the system should be able to adapt	• The system will be able to support more than one manufacturing plant with six months' advance notice. • New versions of the system will be released every six months.

FIGURE 11-14 Operational Requirements

earlier section in this chapter) are very relevant. Consequently, it also includes all of the different types of hardware from mainframe computers to smartphones to IoT devices. Depending on the applications being deployed over the physical architecture, specialized hardware could be required, such as 3D displays, 3D printing, 3D sound systems, and tablets with accelerometers. With today's technology, the possible combinations of hardware that can be used to solve a problem are nearly endless. Consequently, this is one area where additional expertise might be required.

System Integration Requirements *System integration requirements* are those that require the system to operate with other information systems, either inside or outside the company. These typically specify interfaces through which data will be exchanged with other systems.

Portability Requirements Information systems never remain constant. Business needs change and operating technologies change, so the information systems that support them and run on them must change, too. *Portability requirements* define how the technical operating environments might change over time and how the system must respond (e.g., the system currently runs on Windows, whereas in the future the system might have to be deployed on Linux). Portability requirements also refer to potential changes in business requirements that drive technical environment changes. For example, in the future users might want to access a website from their cell phones.

Maintainability Requirements *Maintainability requirements* specify the business requirement changes that can be anticipated. Not all changes are predictable, but some are. For example, suppose a small company has only one manufacturing plant but is anticipating the construction of a second plant in the next five years. All information systems must be written to make it easy to track each plant separately, whether for personnel, budgeting, or inventory systems. The maintainability requirements attempt to anticipate future requirements so that the systems designed today will be easy to maintain if and when those future requirements appear. Maintainability requirements can also define the update cycle for the system, such as the frequency with which new versions will be released.

Performance Requirements

Performance requirements focus on performance issues, such as response time, capacity, and reliability. Figure 11-15 summarizes three key performance requirement areas and provides some examples.

Speed Requirements *Speed requirements* are exactly what they say: How fast should the system operate? First is the *response time* of the system: How long it takes the system to respond to a user request. Although everyone would prefer low response times, with the system responding immediately to each user request, this is not practical. We could design such a system, but it would be expensive. Most users understand that certain parts of a system will respond quickly, whereas others are slower. Actions that are performed locally on the user's computer must be almost immediate (e.g., typing, dragging, and dropping), whereas others that require communicating across a network can have longer response times (e.g., a Web request).

The second aspect of speed requirements is how long it takes transactions in one part of the system to be reflected in other parts. For example, how soon after an order is placed will the items it contained be shown as no longer available for sale to someone else? If the inventory is not updated immediately, then someone else could place an order for the same item, only to find out later it is out of stock. This is especially true when one considers NoSQL database that does not update all copies of the data immediately (see Chapter 9). Or how soon after an order is placed is it sent to the warehouse to be picked from inventory and shipped?

Type of Requirement	Definition	Examples
Speed Requirements	The time within which the system must perform its functions	• Response time must be less than 7 seconds for any transaction over the network. • The inventory database must be updated in real time. • Orders will be transmitted to the factory floor every 30 minutes.
Capacity Requirements	The total and peak number of users and the volume of data expected	• There will be a maximum of 100–200 simultaneous users at peak use times. • A typical transaction will require the transmission of 10K of data.
Availability and Reliability Requirements	The extent to which the system will be available to the users and the permissible failure rate due to errors	• The system will store data on approximately 5,000 customers for a total of about 2 MB of data. • Scheduled maintenance shall not exceed one 6-hour period each month. • The system shall have 99% uptime performance.

FIGURE 11-15 Performance Requirements

Capacity Requirements *Capacity requirements* attempt to predict how many users the system will have to support, both in total and simultaneously. Capacity requirements are important in understanding the size of the databases, the processing power needed, and so on. The most important requirement is usually the peak number of simultaneous users because this has a direct impact on the processing power of the computer(s) needed to support the system.

It is often easier to predict the number of users for internal systems designed to support an organization's own employees than it is to predict the number of users for customer-facing systems, especially those on the Web. How *does* Weather.com estimate the peak number of users who will simultaneously seek weather information? This is as much an art as a science, so often the team provides a range of estimates, with wider ranges used to signal a less-accurate estimate.

Availability and Reliability Requirements *Availability and reliability requirements* focus on the extent to which users can assume that the system will be available for them to use. Although some systems are intended to be used only during the forty-hour work week, some systems are designed to be used by people around the world. For such systems, project team members need to consider how the application can be operated, supported, and maintained *24/7*. This 24/7 requirement means that users might need help or have questions at any time, and a support desk that is available eight hours a day will not be sufficient support. It is also important to consider what reliability is needed in the system. A system that requires high reliability (e.g., a medical device or telephone switch) needs far greater planning and testing than one that does not have such high-reliability needs (e.g., personnel system, Web catalog).

It is more difficult to predict the peaks and valleys in use of the system when the system has a global audience. Typically, applications are backed up on weekends or late evenings when users are no longer accessing the system. Such maintenance activities need to be rethought with global initiatives. For example, what day(s) of the week is considered a "down" day. In different parts of the world, business does not take place every day. In some parts, Friday is sacred; in other parts, it's Saturday or Sunday. Consequently, political and cultural issues (described below and in Chapter 10) can impact the performance requirements. The development of Web interfaces, in particular, has escalated the need for 24/7 support; by

default, the Web can be accessed by anyone at any time. For example, the developers of a Web application for U.S. outdoor gear and clothing retailer Orvis were surprised when the first order after going live came from Japan.

Security Requirements[17]

The primary purpose of the security requirements is actually to increase the trust that the users have with the system and its data. Good IT security protects the information system from disruption and data loss, whether caused by an intentional act (e.g., a hacker, a terrorist attack) or a random event (e.g., disk failure, tornado). In the past, security has been primarily the responsibility of the operations group—the staff responsible for installing and operating security controls, such as firewalls, intrusion-detection systems, and routine backup and recovery operations. However today, developers of new systems must ensure that the system's *security requirements* produce reasonable precautions to prevent problems; system developers are responsible for ensuring security within the information systems themselves. Furthermore, with the newer agile and DevOps approaches to developing systems, security must be at the forefront of any and all decisions. In fact, one of the major issues with the newer agile and DevOps approaches is the speed at which the software is deployed almost forces the developers to ignore security. Consequently, newer versions of DevOps now include security as a major aspect of the system that must be addressed.[18]

Security is an ever-increasing problem in today's Internet-enabled world. Historically, the greatest security threat has come from inside the organization itself. Ever since the early 1980s when the FBI first began keeping computer crime statistics and security firms began conducting surveys of computer crime, organizational employees have perpetrated the vast majority of computer crimes. For years, 80 percent of unauthorized break-ins, thefts, and sabotage have been committed by insiders, leaving only 20 percent to hackers external to the organizations.

In 2001, that changed. Depending on what survey you read, the percentage of incidents attributed to external hackers in 2001 increased to 50 to 70 percent of all incidents, meaning that the greatest risk facing organizations is now from the outside. Although some of this shift may be due to better internal security and better communications with employees to prevent security problems, much of it is simply due to an increase in differing threats, such as denial of service attacks, malware, phishing, ransomware, social engineering, spyware, trojan horses, and many others, by external hackers. Furthermore, given cloud computing and the IoT, security must be taken very seriously and be addressed as the system is being designed instead of simply an add-on that is considered after the system has been implemented.

Developing security requirements usually starts with some assessment of the value of the system and its data. This helps pinpoint extremely important systems so that the operations staff is aware of the risks. Security within systems usually focuses on specifying who can access what data, identifying the need for encryption and authentication, and ensuring the application prevents the spread of viruses (see Figure 11-16).

[17] For more information, see Brett C. Tjaden, *Fundamentals of Secure Computer Systems* (Wilsonville, OR: Franklin, Beedle, and Associates, 2004); for security controls associated with the Sarbanes–Oxley act, see Dennis C. Brewer, *Security Controls for Sarbanes–Oxley Section 404 IT Compliance: Authorization, Authentication, and Access* (Indianapolis: Wiley, 2006); William Stallings, *Effective Cybersecurity: A Guide to Using Best Practices and Standards* (Upper Saddle River, NJ: Pearson Education, 2019); and O. Sami Saydjari, *Engineering Trustworthy Systems: Get Cybersecurity Design Right the First Time* (New York, NY: McGraw-Hill, 2018).

[18] See, for example, Jim Bird, *DevOpsSec: Delivering Secure Software Through Continuous Delivery*, (Sebastopol, CA: O'Reilly Media, 2016); Len Bass, Ingo Weber, and Liming Zhu, *DevOps: A Software Architect's Perspective* (Upper Saddle River, NJ: Pearson Education, 2015); and Gene Kim, Jez Humble, Patrick Debois, and John Willis, *The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations* (Portland, OR: IT Revolution Press, 2016).

Type of Requirement	Definition	Examples
System Value Estimates	Estimated business value of the system and its data	• The system is not mission critical, but a system outage is estimated to cost $50,000 per hour in lost revenue. • A complete loss of all system data is estimated to cost $20 million.
Access Control Requirements	Limitations on who can access what data	• Only department managers will be able to change inventory items within their own department. • Telephone operators will be able to read and create items in the customer file but cannot change or delete items.
Encryption and the Authentication Requirements	Defines what data will be encrypted where and whether authentication will be needed for user access	• Data will be encrypted from the user's computer to website to provide secure ordering. • Users logging in from outside the office will be required to authenticate.
Virus Control Requirements	Requirements to control the spread of viruses	• All uploaded files will be checked for viruses before being saved in the system.

FIGURE 11-16 Security Requirements

System Value The most important computer asset in any organization is not the equipment; it is the organization's data. For example, suppose someone destroyed a mainframe computer worth $10 million. The mainframe could be replaced, simply by buying a new one. It would be expensive, but the problem would be solved in a few weeks. Now suppose someone destroyed all the student records at your university so that no one knew what courses anyone had taken or their grades. The cost would far exceed the cost of replacing a $10 million computer. The lawsuits alone would easily exceed $10 million, and the cost of staff to find paper records and reenter the data from them would be enormous and certainly would take more than a few weeks.

In some cases, the information system itself has value that far exceeds the cost of the equipment as well. For example, for an Internet bank that has no brick and mortar branches, the website is a *mission-critical system*. If the website crashes, the bank cannot conduct business with its customers. A mission-critical application is an information system that is literally critical to the survival of the organization. It is an application that cannot be permitted to fail, and if it does fail, the network staff drops everything else to fix it. Mission-critical applications are usually clearly identified so that their importance is not overlooked.

Even temporary disruptions in service can have significant costs. The costs of disruptions to a company's primary website or the LANs and backbones that support telephone sales operations are often measured in the millions of dollars. Amazon.com, for example, has revenues of more than $10 million per hour, so if its website were unavailable for an hour or even part of an hour, it would lose millions of dollars in revenue. Companies that do less e-business or do telephone sales have lower costs, but recent surveys suggest losses of $100,000 to $200,000 per hour are not uncommon for major customer-facing information systems.

Access Control Requirements Some of the data stored in the system need to be kept confidential; some data need special controls on who is allowed to change or delete it. Personnel records, for example, should be able to be read only by the personnel department and the employee's supervisor; changes should be permitted to be made only by the personnel

department. *Access control requirements* state who can access what data and what type of access is permitted: whether the individual can create, read, update and/or delete the data. The requirements reduce the chance that an authorized user of the system can perform unauthorized actions. The vast majority of these requirements have already been captured in the form of the actors and stakeholders with the real use case descriptions (Chapter 10), the associated classes with the CRC cards (Chapters 5 and 8), and the detailed version of the CRUDE matrix (Chapter 6).

There are several approaches to address access control. An *access control list* is associated with the asset for which you want to control access. For example, a file that you wanted to restrict access to a subset of users would have the users, along with the type of access being granted to them, listed in the access control list for the file. If a user was not in the list, the user would not be able to access the file. Furthermore, if a user was only granted read access, then the user could only read the file. We could actually implement this detailed level of control by capturing the access control lists in the pre-conditions of a method contract (see Chapter 8). Another approach would be to create a *capabilities list* with each user profile. In this case, the access controls would be associated with a specific user instead of a file. With the previous example, the capabilities list would show that the user had read access to the file. Finally, we could combine the access control and capabilities list approaches and require that access is only granted to an IT asset if both the access control list and the capabilities list match. This is the basis for the *access control matrix* approach. However, in terms of processing power requirements, the actual cost of these very low-level approaches may be prohibitive.

Role-based access control attempts to address some of the overhead associated with the access control list, the capability list, and the access control matrix approaches. With *role-based access controls*, access is limited by the role that the user is assigned. So instead of keeping track of the level of access being granted by the individual user, the controls are maintained at the role level. This could be captured with documentation associated with the actors that will be using the system. This is obviously a variation of the capabilities list approach. Furthermore, using operating system file level controls or using controls implemented through the DBMS would provide access controls at a less granular level. These approaches obviously require less processing power and less storage. As usual, it always comes back to cost–benefit analysis of the amount of risk a firm is willing to take on with regard to the cost of security.

Encryption and Authentication Requirements One of the best ways to prevent unauthorized access to data is *encryption*, which is a means of disguising information by the use of mathematical algorithms (or formulas). Encryption can be used to protect data stored in databases or data that are in transit over a network from a database to a computer. There are two fundamentally different types of encryption: symmetric and asymmetric. A *symmetric encryption algorithm* [such as Data Encryption Standard (DES) or Advanced Encryption Standard (AES)] is one in which the key used to encrypt a message is the *same* as the one used to decrypt it, which means that it is essential to protect the key and that a separate key must be used for each person or organization with whom the system shares information (or else everyone can read all the data).

An *asymmetric encryption algorithm* (such as *public key encryption*) is one in which the key used to encrypt data (called the *public key*) is different from the one used to decrypt it (called the *private key*). Even if everyone knows the public key, once the data are encrypted, they cannot be decrypted without the private key. Public key encryption greatly reduces the key-management problem. Each user has its public key that is used to encrypt messages sent to it. These public keys are widely publicized (e.g., listed in a telephone book style directory)—that's why they're called public keys. The private key, in contrast, is kept secret.

Public key encryption also permits *authentication* (or digital signatures). When one user sends a message to another, it is difficult to legally prove who actually sent the message. Legal proof is important in many communications, such as bank transfers and buy/sell orders in currency and stock trading, which normally require legal signatures. Public key encryption algorithms are *invertible*, meaning that text encrypted with either key can be decrypted by the other. Normally, we encrypt with the public key and decrypt with the private key. However, it is possible to do the reverse: encrypt with the private key and decrypt with the public key. Because the private key is secret, only the real user can use it to encrypt a message. Thus, a digital signature or authentication sequence is used as a legal signature on many financial transactions. This signature is usually the name of the signing party plus other unique information from the message (e.g., date, time, or dollar amount). This signature and the other information are encrypted by the sender using the private key. The receiver uses the sender's public key to decrypt the signature block and compares the result to the name and other key contents in the rest of the message to ensure a match.

The only problem with this approach lies in ensuring that the person or organization that sent the document with the correct private key is the actual person or organization. Anyone can post a public key on the Internet, so there is no way of knowing for sure who actually used it. For example, it would be possible for someone other than Organization A in this example to claim to be Organization A when, in fact, he or she is an imposter.

This is where the Internet's public key infrastructure (PKI) becomes important.[19] The PKI is a set of hardware, software, organizations, and polices designed to make public key encryption work on the Internet. PKI begins with a *certificate authority (CA)*, which is a trusted organization that can vouch for the authenticity of the person or organization using authentication (e.g., VeriSign). A person wanting to use a CA registers with the CA and must provide some proof of identify. There are several levels of certification, ranging from a simple confirmation from a valid e-mail address to a complete government-style background check with an in-person interview. The CA issues a digital certificate that is the requestor's public key, encrypted using the CA's private key as proof of identify. This certificate is then attached to the user's e-mail or Web transactions in addition to the authentication information. The receiver then verifies the certificate by decrypting it with the CA's public key and must also contact the CA to ensure that the user's certificate has not been revoked by the CA.

The encryption and authentication requirements state what encryption and authentication requirements are needed for what data. For example, will sensitive data such as customer credit-card numbers be stored in the database in encrypted form, or will encryption be used to take orders over the Internet from the company's website? Will users be required to use a digital certificate in addition to a standard password?

Virus Control Requirements *Virus control requirements* address the single most common security problem: *viruses*. Studies have shown that almost 90 percent of organizations suffer a virus infection each year. Viruses cause unwanted events—some harmless (such as nuisance messages), some serious (such as the destruction of data). Any time a system permits data to be imported or uploaded from a user's computer, there is the potential for a virus infection. Many systems require that all information systems that permit the import or upload of user files to check those files for viruses before they are stored in the system.

Cultural and Political Requirements

Cultural and political requirements are those specific to the countries in which the system will be used. In today's global business environment, organizations are expanding their systems to

[19] For more on the PKI, see http://datatracker.ietf.org/wg/pkix/charter/.

reach users around the world. Although this can make great business sense, its impact on application development should not be underestimated. Yet another important part of the design of the system's physical architecture is understanding the global cultural and political requirements for the system (see Chapter 10 and Figure 11-17).

Customization Requirements For global applications, the project team needs to give some thought to *customization requirements:* How much of the application will be controlled by a central group, and how much of the application will be managed locally? For example, some companies allow subsidiaries in some countries to customize the application by omitting or adding certain features. This decision has trade-offs between flexibility and control because customization often makes it more difficult for the project team to create and maintain the application. It also means that training can differ among different parts of the organization, and customization can create problems when staff moves from one location to another.

Owing to the use of different languages, in some cases, specialized hardware that has been customized to the local culture is required. For example, having specialized keyboards makes sense for any language that does not use the typical Roman alphabet, e.g., Arabic, Hebrew, Greek, Japanese, Korean, Mandarin, or Russian. There are also emulators available for many different languages. Depending on the users being served, assistive devices could be required, such as Braille devices, eye-tracking devices, head pointers, head/mouth stick keyboards, or adaptive ability switches. Depending on the cultural and political requirements, many different hardware platforms might need to be considered.

Legal Requirements *Legal requirements* are requirements imposed by laws and government regulations. System developers sometimes forget to think about legal regulations; unfortunately, forgetting comes at some risk because ignorance of the law is no defense. For example, in 1997 a French court convicted the Georgia Institute of Technology of violating French language law. Georgia Tech operated a small campus in France that offered summer programs for American students. The information on the campus Web server was primarily in English because classes are conducted in English, which violated the law requiring French to be the predominant language on all Internet servers in France. By formally considering legal regulations, you are less likely to overlook them. Another major example is the recent European court ruling regarding the user's right to be forgotten.

Type of Requirement	Definition	Examples
Customization Requirements	Specification of what aspects of the system can be changed by local users	• Country managers will be able to define new fields in the product database to capture country-specific information. • Country managers will be able to change the format of the telephone number field in the customer database.
Legal Requirements	The laws and regulations that impose requirements on the system	• Personal information about customers cannot be transferred out of European Union countries into the United States. • It is against U.S. federal law to divulge information on who rented what videotape, so access to a customer's rental history is permitted only to regional managers.

FIGURE 11-17 Cultural and Political Requirements

Another area that legal requirements can cause problems is with the *end-user license agreements*. When is the last time that you carefully have read the end user license agreement when you installed software on your machine? From an organizational perspective, this could cause problems. Some agreements commit the organization to allow access to the data that is processed by the software. Other agreements require that the organization be bound by the laws of a foreign country. Or, with a federal government system such as in the United States, the agreement could require that the organization to follow the laws of another state. So, from an organization perspective, the organization should restrict the installation of any software until the organization's lawyers clear it. Otherwise, a user could be granting access to an outside organization to organizational data and possibly to intellectual property. In other words, we need to have the "fine print" of the agreement read by an expert to prevent the accidental loss of organizational assets.

Synopsis

In many cases, the technical environment requirements as driven by the business requirements can simply define the physical architecture layer. In this case, the choice is simple: Business requirements dominate other considerations. For example, the business requirements might specify that the system needs to work over the Web using the customer's Web browser. In this case, the architecture probably should be a thin client–server. Such business requirements are most likely in systems designed to support external customers. Internal systems can also impose business requirements, but usually they are not as restrictive.

In the event that the technical environment requirements do not stipulate a specific architecture, then the other nonfunctional requirements become important. Even in cases when the business requirements drive the architecture, it is still important to work through and refine the remaining nonfunctional requirements because they are important in later stages of design and implementation.

VERIFYING AND VALIDATING THE PHYSICAL ARCHITECTURE LAYER

Like the models on the other layers, the infrastructure design and the hardware and software specifications of the physical architecture layer need to be verified and validated. Verifying and validating the design of the physical architecture layer fall into three basic groups.

First, we recommend verifying and validating deployment diagrams by ensuring that all of them are in fact consistent and balanced. For example, each of the nodes in a top-level network model deployment diagram should be associated with a separate deployment diagram that represents the low-level network model for the node.

Second, the hardware and software specifications should be consistent with the "lowest-level" network models. For example, if a low-level network model for an office describes a set of workstations, servers, printers, switches, routers, etc., then the hardware and software specification for that location should be the details for each of the IT artifacts for that location.

Third, once the system has been implemented, testing of the nonfunctional requirements becomes crucial. In this case, tests must be designed and performed for each of the nonfunctional requirements. For example, for the performance requirements, load testing must be performed to identify possible performance bottlenecks in the network. We will return to this topic in Chapter 12.